1. Implementation of Basic Array Operations (Insertion, Deletion and Traversal)

AIM: To write a C program to perform insertion, deletion and traversal operations on an array.

ALGORITHM:

1. Start the program and read the number of elements and array elements.
2. Display the menu: insert, delete, traverse or exit.
3. For insertion, read the position and value, shift elements right and insert the value.
4. For deletion, read the position, shift the following elements left and reduce the size.
5. Display the array and repeat until the user selects exit.

PROGRAM:

```
#include <stdio.h>
int main() {
   int a[100], n, i, ch, pos, val;
   printf("Enter number of elements: ");
   scanf("%d", &n);
   printf("Enter elements: ");
   for(i = 0; i < n; i++) scanf("%d", &a[i]);

   do {
      printf("\n1.Insert  2.Delete  3.Traverse  4.Exit\n");
      scanf("%d", &ch);
      if(ch == 1) {
         printf("Enter position and value: ");
         scanf("%d%d", &pos, &val);
         if(pos < 1 || pos > n + 1 || n == 100)
            printf("Invalid position\n");
         else {
            for(i = n; i >= pos; i--) a[i] = a[i - 1];
            a[pos - 1] = val;
            n++;
         }
      } else if(ch == 2) {
         printf("Enter position: ");
         scanf("%d", &pos);
         if(pos < 1 || pos > n) printf("Invalid position\n");
         else {
            for(i = pos - 1; i < n - 1; i++) a[i] = a[i + 1];
            n--;
         }
      } else if(ch == 3) {
         printf("Array: ");
         for(i = 0; i < n; i++) printf("%d ", a[i]);
         printf("\n");
      }
   } while(ch != 4);
   return 0;
}
```

OUTPUT: Example: Initial array: 10 20 30. Insert 15 at position 2. Traversal displays: 10 15 20 30. Delete position 3. Traversal displays: 10 15 30.

RESULT: Thus, insertion, deletion and traversal operations on an array were implemented successfully.

2. Dynamic Employee Record System using Structure Pointers

AIM: To write a C program to store and display employee details using a structure pointer.

ALGORITHM:

6. Define a structure named Employee with ID, name and salary.
7. Declare a structure pointer and allocate memory using malloc().
8. Read the employee details through the pointer.
9. Display the stored employee details.
10. Free the allocated memory and stop the program.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
struct Employee {
   int id;
   char name[30];
   float salary;
};
int main() {
   struct Employee *e;
   e = (struct Employee *)malloc(sizeof(struct Employee));
   if(e == NULL) {
      printf("Memory allocation failed");
      return 1;
   }
   printf("Enter employee ID: ");
   scanf("%d", &e->id);
   printf("Enter employee name: ");
   scanf("%29s", e->name);
   printf("Enter salary: ");
   scanf("%f", &e->salary);

   printf("\nEmployee Details\n");
   printf("ID: %d\nName: %s\nSalary: %.2f\n",
        e->id, e->name, e->salary);
   free(e);
   return 0;
}
```

OUTPUT: Sample Output: ID: 101; Name: Arun; Salary: 25000.00. The entered employee details are displayed.

RESULT: Thus, employee details were stored and displayed using structure pointers successfully.

3. Implementation of Stack and Expression Conversion (Infix to Postfix)

AIM: To write a C program to convert an infix expression containing single-letter or single-digit operands into postfix form using a stack.

ALGORITHM:

11. Read the infix expression from the user.
12. If the symbol is an operand, add it to the postfix expression.
13. If it is an opening bracket, push it onto the stack.
14. If it is an operator, pop operators of higher or equal precedence and then push the current operator.
15. Pop the remaining operators and display the postfix expression.

PROGRAM:

```
#include <stdio.h>
#include <ctype.h>
char st[100];
int top = -1;
int prec(char c) {
   if(c == '^') return 3;
   if(c == '*' || c == '/') return 2;
   if(c == '+' || c == '-') return 1;
   return 0;
}
int main() {
   char in[100], post[100], c;
   int i, k = 0;
   printf("Enter infix expression: ");
   scanf("%99s", in);

   for(i = 0; in[i] != '\0'; i++) {
      c = in[i];
      if(isalnum((unsigned char)c)) post[k++] = c;
      else if(c == '(') st[++top] = c;
      else if(c == ')') {
         while(top != -1 && st[top] != '(') post[k++] = st[top--];
         if(top != -1) top--;
      } else {
         while(top != -1 && st[top] != '(' &&
               prec(st[top]) >= prec(c))
            post[k++] = st[top--];
         st[++top] = c;
      }
   }
   while(top != -1) post[k++] = st[top--];
   post[k] = '\0';
   printf("Postfix expression: %s\n", post);
   return 0;
}
```

OUTPUT: Sample Output: Enter infix expression: A+B*C. Postfix expression: ABC*+.

RESULT: Thus, the infix expression was converted into postfix form using a stack successfully.

4. Postfix Expression Evaluation using Stack

AIM: To write a C program to evaluate a postfix expression containing single-digit operands and operators +, -, * and /.

ALGORITHM:

16. Read the postfix expression.
17. If the symbol is a digit, convert it to an integer and push it onto the stack.
18. If the symbol is an operator, pop the second operand and then the first operand.
19. Perform the operation and push the result onto the stack.
20. After scanning the expression, display the value at the top of the stack.

PROGRAM:

```
#include <stdio.h>
#include <ctype.h>
int main() {
   char exp[100], c;
   int st[100], top = -1, i, a, b;
   printf("Enter postfix expression: ");
   scanf("%99s", exp);

   for(i = 0; exp[i] != '\0'; i++) {
      c = exp[i];
      if(isdigit((unsigned char)c))
         st[++top] = c - '0';
      else {
         b = st[top--];
         a = st[top--];
         switch(c) {
            case '+': st[++top] = a + b; break;
            case '-': st[++top] = a - b; break;
            case '*': st[++top] = a * b; break;
            case '/': st[++top] = a / b; break;
         }
      }
   }
   printf("Result = %d\n", st[top]);
   return 0;
}
```

OUTPUT: Sample Output: Enter postfix expression: 23*54*+. Result = 26.

RESULT: Thus, the given postfix expression was evaluated using a stack successfully.

5. Implementation of Simple Queue using Array

AIM: To write a C program to implement enqueue, dequeue and display operations in a simple linear queue using an array.

ALGORITHM:

21. Initialize front and rear to -1.

22. For enqueue, check overflow and insert the new element at rear.

23. If the queue is empty, set front to 0.

24. For dequeue, check underflow, display the front element and increment front.

25. Display the queue elements from front to rear.

PROGRAM:

```
#include <stdio.h>
#define MAX 5
int main() {
   int q[MAX], front = -1, rear = -1, ch, x, i;
   do {
      printf("\n1.Enqueue 2.Dequeue 3.Display 4.Exit\n");
      scanf("%d", &ch);
      if(ch == 1) {
         if(rear == MAX - 1) printf("Queue is full\n");
         else {
            printf("Enter value: ");
            scanf("%d", &x);
            if(front == -1) front = 0;
            q[++rear] = x;
         }
      } else if(ch == 2) {
         if(front == -1 || front > rear) printf("Queue is empty\n");
         else {
            printf("Deleted: %d\n", q[front++]);
            if(front > rear) front = rear = -1;
         }
      } else if(ch == 3) {
         if(front == -1) printf("Queue is empty\n");
         else {
            for(i = front; i <= rear; i++) printf("%d ", q[i]);
            printf("\n");
         }
      }
   } while(ch != 4);
   return 0;
}
```

OUTPUT: Sample Output: Enqueue 10, 20, 30. Display: 10 20 30. Dequeue: Deleted 10.

RESULT: Thus, enqueue, dequeue and display operations of a simple queue were implemented successfully.

6. Implementation of Priority Queue using Unordered Array

AIM: To write a C program to implement a priority queue using an unordered array. A smaller priority number represents a higher priority.

ALGORITHM:

26. Initialize the queue with zero elements.

27. For insertion, read the value and its priority and add them at the end.

28. Do not arrange elements during insertion; hence the array remains unordered.

29. For deletion, find the element with the smallest priority number.

30. Delete that element by shifting the remaining elements and display the queue.

PROGRAM:

```
#include <stdio.h>
#define MAX 20
int main() {
   int value[MAX], pri[MAX], n = 0, ch, x, p, i, k;
   do {
      printf("\n1.Insert 2.Delete 3.Display 4.Exit\n");
      scanf("%d", &ch);
      if(ch == 1) {
         if(n == MAX) printf("Queue is full\n");
         else {
            printf("Enter value and priority: ");
            scanf("%d%d", &x, &p);
            value[n] = x;
            pri[n] = p;
            n++;
         }
      } else if(ch == 2) {
         if(n == 0) printf("Queue is empty\n");
         else {
            k = 0;
            for(i = 1; i < n; i++)
               if(pri[i] < pri[k]) k = i;
            printf("Deleted element: %d\n", value[k]);
            for(i = k; i < n - 1; i++) {
               value[i] = value[i + 1];
               pri[i] = pri[i + 1];
            }
            n--;
         }
      } else if(ch == 3) {
         if(n == 0) printf("Queue is empty\n");
         for(i = 0; i < n; i++)
            printf("Value=%d Priority=%d\n", value[i], pri[i]);
      }
   } while(ch != 4);
   return 0;
}
```

OUTPUT: Sample Output: Insert (25, priority 2), (10, priority 1). Delete: Deleted element: 10.

RESULT: Thus, a priority queue was implemented using an unordered array successfully.

7. Implementation of Singly Linked List (Insertion, Deletion and Traversal)

AIM: To write a C program to perform insertion, deletion and traversal operations in a singly linked list.

ALGORITHM:

31. Create a node containing data and a next pointer.

32. For insertion, create a new node and link it at the beginning.

33. For deletion, remove the first node and update the head pointer.

34. For traversal, start at head and display each node until NULL.

35. Repeat the menu operations until exit is selected.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
   int data;
   struct Node *next;
};
int main() {
   struct Node *head = NULL, *temp, *newnode;
   int ch, x;
   do {
      printf("\n1.Insert 2.Delete 3.Display 4.Exit\n");
      scanf("%d", &ch);
      if(ch == 1) {
         newnode = (struct Node *)malloc(sizeof(struct Node));
         printf("Enter value: ");
         scanf("%d", &x);
         newnode->data = x;
         newnode->next = head;
         head = newnode;
      } else if(ch == 2) {
         if(head == NULL) printf("List is empty\n");
         else {
            temp = head;
            head = head->next;
            printf("Deleted: %d\n", temp->data);
            free(temp);
         }
      } else if(ch == 3) {
         temp = head;
         while(temp != NULL) {
            printf("%d -> ", temp->data);
            temp = temp->next;
         }
         printf("NULL\n");
      }
   } while(ch != 4);
   while(head != NULL) {
      temp = head;
      head = head->next;
```

```
    free(temp);
  }
  return 0;
}
```

OUTPUT: Sample Output: Insert 10 and 20. Display: 20 -> 10 -> NULL. Delete: 20.

RESULT: Thus, insertion, deletion and traversal operations in a singly linked list were implemented successfully.

8. Implementation of Circular Queue using Linked List

AIM: To write a C program to implement enqueue, dequeue and display operations in a circular queue using a linked list.

ALGORITHM:

36. Initially set front and rear to NULL.

37. For enqueue, create a new node and link it after rear.

38. Make the new node point to front and update rear.

39. For dequeue, remove the front node and update front; if it was the only node, set both pointers to NULL.

40. Display nodes from front until front is reached again.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
   int data;
   struct Node *next;
};
int main() {
   struct Node *front = NULL, *rear = NULL, *p, *n;
   int ch, x;
   do {
      printf("\n1.Enqueue 2.Dequeue 3.Display 4.Exit\n");
      scanf("%d", &ch);
      if(ch == 1) {
         n = (struct Node *)malloc(sizeof(struct Node));
         printf("Enter value: ");
         scanf("%d", &x);
         n->data = x;
         if(front == NULL) front = rear = n;
         else {
            rear->next = n;
            rear = n;
         }
         rear->next = front;
      } else if(ch == 2) {
         if(front == NULL) printf("Queue is empty\n");
         else {
            p = front;
            printf("Deleted: %d\n", p->data);
            if(front == rear) front = rear = NULL;
            else {
               front = front->next;
               rear->next = front;
            }
            free(p);
         }
      } else if(ch == 3) {
         if(front == NULL) printf("Queue is empty\n");
```

```
        else {
            p = front;
            do {
                printf("%d ", p->data);
                p = p->next;
            } while(p != front);
            printf("\n");
        }
    }
  } while(ch != 4);
  while(front != NULL) {
    p = front;
    if(front == rear) front = rear = NULL;
    else {
        front = front->next;
        rear->next = front;
    }
    free(p);
  }
  return 0;
}
```

OUTPUT: Sample Output: Enqueue 10, 20, 30. Display: 10 20 30. Dequeue: Deleted 10.

RESULT: Thus, a circular queue using a linked list was implemented successfully.

9. Binary Search Tree Implementation and Traversals

AIM: To write a C program to create a Binary Search Tree and perform inorder, preorder and postorder traversals.

ALGORITHM:

41. Create a node with data, left pointer and right pointer.

42. Insert each value into the tree: smaller values go left and larger values go right.

43. Inorder traversal visits left, root and right.

44. Preorder traversal visits root, left and right.

45. Postorder traversal visits left, right and root; display all traversals.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
   int data;
   struct Node *left, *right;
};
struct Node *insert(struct Node *root, int x) {
   if(root == NULL) {
      root = (struct Node *)malloc(sizeof(struct Node));
      root->data = x;
      root->left = root->right = NULL;
   } else if(x < root->data)
      root->left = insert(root->left, x);
   else if(x > root->data)
      root->right = insert(root->right, x);
   return root;
}
void inorder(struct Node *r) {
   if(r) { inorder(r->left); printf("%d ", r->data); inorder(r->right); }
}
void preorder(struct Node *r) {
   if(r) { printf("%d ", r->data); preorder(r->left); preorder(r->right); }
}
void postorder(struct Node *r) {
   if(r) { postorder(r->left); postorder(r->right); printf("%d ", r->data); }
}
int main() {
   struct Node *root = NULL;
   int n, x, i;
   printf("Enter number of nodes: ");
   scanf("%d", &n);
   printf("Enter values: ");
   for(i = 0; i < n; i++) {
      scanf("%d", &x);
      root = insert(root, x);
   }
   printf("Inorder: "); inorder(root);
   printf("\nPreorder: "); preorder(root);
   printf("\nPostorder: "); postorder(root);
```

```
    printf("\n");
    return 0;
}
```

OUTPUT: Sample Output: Input values: 50 30 70 20 40. Inorder: 20 30 40 50 70. Preorder: 50 30 20 40 70. Postorder: 20 40 30 70 50.

RESULT: Thus, a Binary Search Tree was created and its inorder, preorder and postorder traversals were performed successfully.

10. Implementation of Graph Search Algorithms (BFS and DFS)

AIM: To write a C program to perform Breadth First Search (BFS) and Depth First Search (DFS) on a graph represented using an adjacency matrix.

ALGORITHM:

46. Read the number of vertices and the adjacency matrix.

47. Read the starting vertex.

48. For BFS, insert the starting vertex into a queue and mark it visited.

49. Remove vertices from the queue, display them and add their unvisited adjacent vertices.

50. For DFS, use recursion to visit and display each unvisited adjacent vertex.

PROGRAM:

```
#include <stdio.h>
int a[20][20], visited[20], n;
void dfs(int v) {
   int i;
   visited[v] = 1;
   printf("%d ", v);
   for(i = 0; i < n; i++)
      if(a[v][i] && !visited[i]) dfs(i);
}
void bfs(int start) {
   int q[20], f = 0, r = 0, v, i;
   for(i = 0; i < n; i++) visited[i] = 0;
   q[r++] = start;
   visited[start] = 1;
   while(f < r) {
      v = q[f++];
      printf("%d ", v);
      for(i = 0; i < n; i++)
         if(a[v][i] && !visited[i]) {
            q[r++] = i;
            visited[i] = 1;
         }
   }
}
int main() {
   int i, j, start;
   printf("Enter number of vertices: ");
   scanf("%d", &n);
   printf("Enter adjacency matrix:\n");
   for(i = 0; i < n; i++)
      for(j = 0; j < n; j++) scanf("%d", &a[i][j]);
   printf("Enter starting vertex (0 to %d): ", n - 1);
   scanf("%d", &start);
   printf("BFS: "); bfs(start);
   for(i = 0; i < n; i++) visited[i] = 0;
   printf("\nDFS: "); dfs(start);
   printf("\n");
   return 0;
}
```

OUTPUT: Sample Output (for a connected graph): BFS: 0 1 2 3. DFS: 0 1 3 2. The exact order depends on the adjacency matrix.
RESULT: Thus, BFS and DFS graph traversal algorithms were implemented successfully.

11. Implementation of Bubble, Selection and Heap Sort

AIM: To write a C program to sort a list of integers using Bubble Sort, Selection Sort and Heap Sort.

ALGORITHM:

51. Read the number of elements and the array values.

52. Bubble Sort compares adjacent elements and swaps them if they are in the wrong order.

53. Selection Sort finds the smallest element and places it in the correct position.

54. Heap Sort builds a max heap and repeatedly moves the largest element to the end.

55. Display the sorted arrays for all three sorting methods.

PROGRAM:

```
#include <stdio.h>
void print(int a[], int n) {
   int i;
   for(i = 0; i < n; i++) printf("%d ", a[i]);
   printf("\n");
}
void bubble(int a[], int n) {
   int i, j, t;
   for(i = 0; i < n - 1; i++)
      for(j = 0; j < n - i - 1; j++)
         if(a[j] > a[j + 1]) {
            t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
         }
}
void selection(int a[], int n) {
   int i, j, min, t;
   for(i = 0; i < n - 1; i++) {
      min = i;
      for(j = i + 1; j < n; j++)
         if(a[j] < a[min]) min = j;
      t = a[i]; a[i] = a[min]; a[min] = t;
   }
}
void heapify(int a[], int n, int i) {
   int largest = i, l = 2 * i + 1, r = 2 * i + 2, t;
   if(l < n && a[l] > a[largest]) largest = l;
   if(r < n && a[r] > a[largest]) largest = r;
   if(largest != i) {
      t = a[i]; a[i] = a[largest]; a[largest] = t;
      heapify(a, n, largest);
   }
}
void heap(int a[], int n) {
   int i, t;
   for(i = n / 2 - 1; i >= 0; i--) heapify(a, n, i);
   for(i = n - 1; i > 0; i--) {
      t = a[0]; a[0] = a[i]; a[i] = t;
      heapify(a, i, 0);
   }
```

```
}
int main() {
   int a[50], b[50], c[50], d[50], n, i;
   printf("Enter number of elements: ");
   scanf("%d", &n);
   printf("Enter elements: ");
   for(i = 0; i < n; i++) {
      scanf("%d", &a[i]);
      b[i] = c[i] = d[i] = a[i];
   }
   bubble(b, n);
   selection(c, n);
   heap(d, n);
   printf("Bubble Sort: "); print(b, n);
   printf("Selection Sort: "); print(c, n);
   printf("Heap Sort: "); print(d, n);
   return 0;
}
```

OUTPUT: Sample Output: Input: 5 4 1 3 2. Bubble Sort: 1 2 3 4 5. Selection Sort: 1 2 3 4 5. Heap Sort: 1 2 3 4 5.

RESULT: Thus, Bubble Sort, Selection Sort and Heap Sort were implemented successfully.

12. Implementation of Hashing with Collision Resolution using Linear Probing

AIM: To write a C program to implement hashing with collision resolution using linear probing.

ALGORITHM:

56. Initialize the hash table with empty positions.

57. Read the keys to be inserted.

58. Calculate the hash index using key modulo table size.

59. If the position is occupied, check the next position sequentially.

60. Insert the key in the first empty position and display the hash table.

PROGRAM:

```
#include <stdio.h>
#define SIZE 10
int main() {
   int h[SIZE], n, key, index, i, j;
   for(i = 0; i < SIZE; i++) h[i] = -1;
   printf("Enter number of keys: ");
   scanf("%d", &n);
   printf("Enter keys: ");
   for(i = 0; i < n; i++) {
      scanf("%d", &key);
      index = key % SIZE;
      j = 0;
      while(j < SIZE && h[(index + j) % SIZE] != -1) j++;
      if(j == SIZE) printf("Hash table is full; key %d not inserted\n", key);
      else h[(index + j) % SIZE] = key;
   }
   printf("Hash Table:\n");
   for(i = 0; i < SIZE; i++)
      printf("%d : %d\n", i, h[i]);
   return 0;
}
```

OUTPUT: Sample Output: Keys: 12 22 32. They are placed at indexes 2, 3 and 4 respectively by linear probing.

RESULT: Thus, hashing with collision resolution using linear probing was implemented successfully.